Beursrally 2026: Maximising Expected Prize Money in a Stock-Picking Competition

Jasper Geens

November 2026

Contents

Abstract . . . 2
1. The game and the objective . . . 3
The rules that matter . . . 3
Why not maximise return . . . 3
2. System overview . . . 4
3. Data: free sources, strict timing . . . 5
Sources . . . 5
Cleaning (`clean.py`) . . . 6
Features (`features.py`) . . . 6
Leakage tests . . . 7
4. Risk model: how much will each stock move? . . . 7
What is forecast . . . 7
Candidate models . . . 7
How models were judged . . . 8
The choice . . . 9
Correlations . . . 9
5. Return model: a small, honest edge . . . 10
What is forecast . . . 10
Six candidates . . . 10
Walk-forward with embargo . . . 11
Rank IC, not R^2 . . . 11
Choosing and shrinking . . . 12
Why it is used even though it is small . . . 12
6. Scenario generator: 20,000 possible competitions . . . 12
Building one future . . . 12
Fat tails without a fat-tailed distribution . . . 13
A cap on fantasy . . . 14
Speed . . . 14
7. Game engine: the rules, exactly . . . 14
One trading day in the engine . . . 14
What is enforced . . . 14
Strategies as policies . . . 15
8. The competitor field: who are we playing against? . . . 15
Five player types . . . 15
Six fields . . . 16
Prize allocation . . . 16
What it does not model . . . 17
9. The starting portfolio: search on one set, judge on another . . . 17

The search problem . . . 17
Twenty starting points, in parallel . . . 17
The control set: no winner's curse . . . 17
Candidates and exclusions . . . 18
Is the search better than luck? . . . 18
Robust to the type of field? . . . 18
10. Trading during the competition: act only on confirmed gains . . . 19
First attempt: fixed rules (rejected) . . . 19
The rollout . . . 19
The winner's curse, and the fix . . . 19
The weekly routine with the real standings . . . 20
11. Validation: choose, check, and test once . . . 20
The protocol . . . 20
Volatility . . . 21
Returns . . . 21
Scenarios . . . 22
12. Results: the model bets against the crowd . . . 22
Two very different answers . . . 22
Why a calmer portfolio wins more . . . 22
What the bet depends on . . . 23
13. Limitations, and what was left out on purpose . . . 23
Known limitations . . . 23
Deliberately left out . . . 23
14. Software, packages and outputs . . . 24
Packages, and what each one does here . . . 24
Project layout . . . 25
What the system produces . . . 26
Appendix A. Script reference . . . 26
Data . . . 26
Risk . . . 27
Returns . . . 28
Scenarios . . . 28
Game, strategy and weekly routine . . . 29
Tests . . . 29
Appendix B. Questions you can expect, with short answers . . . 30

Abstract

Beursrally is a nine-week virtual stock competition run by the Belgian newspaper De Tijd: every player starts with €50,000, picks from about 370 European and US securities, and only the top 3 of each ranking win money. That last rule changes everything. A strategy that maximises expected return is almost certain to finish somewhere in the anonymous middle, where it earns exactly as much as last place: nothing.

This project therefore optimises **expected prize money** directly. It forecasts how much each stock will move (a volatility model validated out of sample), adds a small forecast of which stocks will do better than average (a return model), and turns both into 20,000 correlated Monte Carlo futures up to the last day of the competition, 22 January 2027. In every future it plays the exact game rules for our portfolio and for a simulated field of tens of thousands of competitors, and counts what we would win. A search over portfolios picks the one with the highest average prize; during the competition a weekly rollout re-evaluates the real standings, with the real portfolios of

the leaders, and trades only when a change is confirmed on a second, independent set of futures.

Three results stand out. The volatility model beats the standard industry benchmark at every horizon in a final test on 16 months of unseen data. The scenario generator reproduces the distribution of real outcomes, including the best stock of each period, far better than a normal distribution. And the optimal starting portfolio is not the most volatile one but a **contrarian** one: because the field crowds into AI and chip stocks, the model prefers volatile stocks that tend to move against that crowd, which more than doubled the expected prize money compared with simply buying the ten most volatile stocks (€317 vs €135 in the same simulated futures).

1. The game and the objective

The objective is to maximise expected prize money, because the prize structure rewards only extreme outcomes and pays nothing for being good.

The rules that matter

Rule	Value	Why it matters for the model
Starting capital	€50,000 virtual cash	everyone starts equal
Period	16 November 2026 – 22 January 2027, 48 trading days	short horizon: luck dominates skill
Universe	445 listed products: 354 stocks (mostly Euronext, plus US), 15 ETFs and 69 funds; the model uses the 369 stocks and ETFs	150 are US-listed and follow US holidays
Position cap	max 10% of the portfolio value, checked at purchase only	forces at least 10 positions; a winner may grow beyond 10%
Orders	max 5 executed orders per day; per security max 1 buy and 1 sell per day	building a portfolio takes two days
Costs	€5 + 0.35% per order; no selling costs at the end	trading is expensive: a round trip costs ~0.9%
Shares	whole shares only; cash can never go negative	a €880 share leaves cash idle
Quiz	€100 per correct daily answer (46 quiz days assumed)	up to €4,600 of risk-free gain, about +9%
Finance Avenue	€500 for attending the event on 28 November, credited 4 December	another risk-free +1%
Prizes	general ranking: €10,000 / €2,500 / €1,500; student ranking: €2,500 / €1,500 / €1,000	only the top 3 earn anything
One prize per person	a student in the general top 3 takes the highest prize only; the next student moves up	handled exactly in the prize allocation

Why not maximise return

Last year about 34,000 people played; the trial round this year had 9,736 players, 1,684 of them students. To finish third among thousands you need an outcome far out in the right tail, while

the expected return of any sensible portfolio over nine weeks is a few percent. So the question is not "which portfolio grows most on average" but "which portfolio has the best chance of landing in the extreme right tail, *ahead of everyone else who is also trying*".

This has three consequences that drive every design choice below:

1. **Variance is good.** A calm portfolio cannot reach the top 3. Volatility is what creates the chance of an extreme outcome.
2. **Correlation with the field matters as much as correlation within the portfolio.** If thousands of players hold the same stocks as you, their outcomes move with yours and you cannot pass them. You win in the futures where *your* bets pay off and *theirs* don't.
3. **Rank is what counts, not value.** Finishing 4th pays the same as finishing 34,000th. So the model needs the full distribution of outcomes, for us and for the competitors, not just averages.

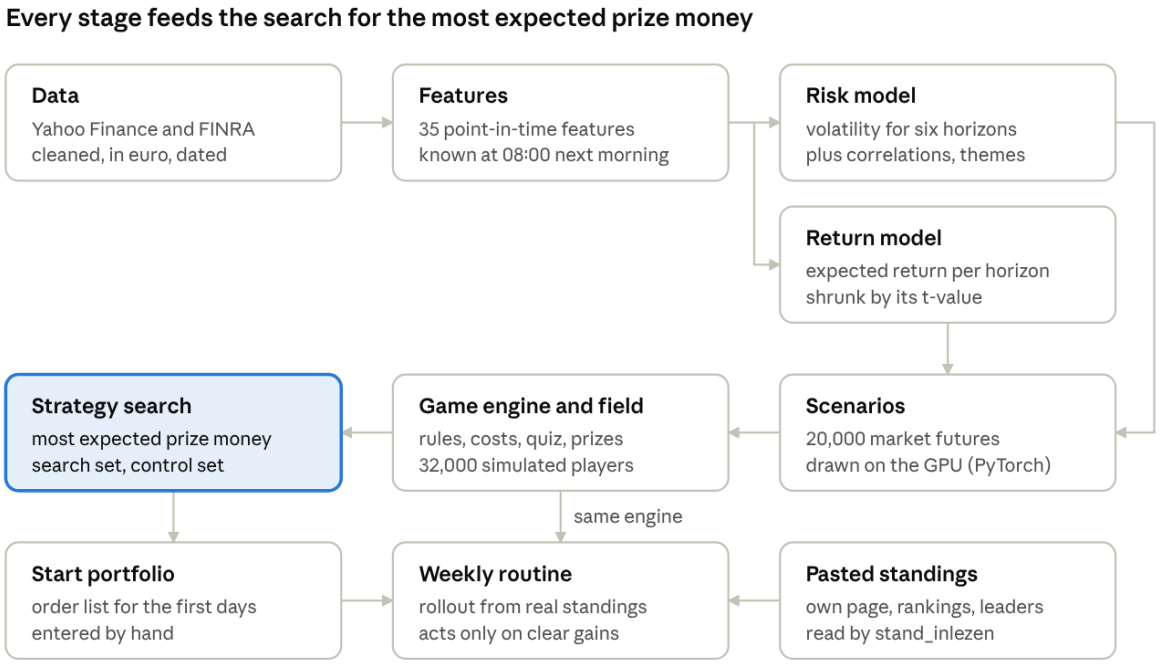
The quantity optimised throughout is therefore

$$E[\text{prize}] = \sum_{k \in \{G1, G2, G3, S1, S2, S3\}} \text{prize}_k \cdot P(\text{we hold place } k)$$

estimated by simulation: generate many possible futures, play the game in each, and average what we win.

2. System overview

The system is a pipeline of stages, each of which reads the files of the stage before it and writes its own. Read the diagram from the top left, then follow the arrows down the right side and back to the left.



The pipeline, from raw data to orders, and the weekly loop.

Forecasting (top right). The data stage downloads prices and other inputs and cleans them. The feature stage turns them into 35 numbers per stock per day, each stamped with the moment it would have been known. Two models then make forecasts from those features: the risk model says how much each stock is likely to move, and how stocks move together; the return model says, very cautiously, which stocks are likely to do better than others.

Simulation (middle). The scenario generator turns those forecasts into 20,000 possible futures of the whole market over the nine weeks of the competition. The game engine plays the competition in each of them under the real rules (order limits, costs, quiz money, prizes), together with a field of 32,000 simulated players.

Decision (bottom left). The strategy search tries thousands of portfolios in those simulated competitions and keeps the one with the highest average prize money, checked on a second, separate set of scenarios. Its output is an order list, entered by hand on the competition site.

The weekly loop (bottom). Every week, the real standings are copied from the competition pages and read in automatically. The rollout then reruns the same engine from the real situation, with the real leaders in the field, and proposes a change only when it clearly beats doing nothing.

The forecasting stages run before the start and are refreshed with new data during the competition; the strategy search runs once; the weekly loop runs every weekend. Nothing in the pipeline places orders or reads the competition site by itself.

3. Data: free sources, strict timing

Everything runs on free public data, and every number a model sees is stamped with the moment it became known, so that no forecast can peek at the future.

Sources

Source	What	How often
Yahoo Finance (`yfinance`)	daily open, high, low, close, volume per security, about 8–10 years; splits and dividends; earnings dates and surprises; shares outstanding	prices daily, the rest weekly
Yahoo Finance	option chains of US stocks around the price, expiries around 22 January 2027	snapshot per day
Yahoo Finance	market series: S&P 500, Nasdaq 100, STOXX 600, MSCI World, VIX, VXN, bitcoin, EUR/USD, US interest rates	daily
FINRA	short interest of US stocks, twice a month since June 2021	per settlement date

Prices are adjusted for **splits only**, not for dividends (Yahoo's "Adj Close" is not used). That matches the game: Beursrally values a portfolio at the quoted price. All prices are converted to euro with the EUR/USD rate of the same day, because the ranking is in euro and a US stock's euro return includes the currency move.

Cleaning (`clean.py`)

Raw market data is full of traps that look like enormous returns. The cleaning step removes them before any model sees them:

1. **Incomplete days**: rows fetched during trading hours (no close yet) are dropped.
2. **Known artefacts** from `config/artefacten.csv`: e.g. Stellantis only from its listing in January 2021 (earlier prices belong to a predecessor), and the TomTom capital return plus share consolidation in May 2019 (a jump that is entirely overnight).
3. **Fake zero days**: days without volume and with an unchanged price are not real trading and would create artificial 0% returns.
4. **The first 10 trading days after an IPO**, whose price discovery is not representative.
5. **Returns per security on its own trading days**: a Belgian stock does not get a 0% return on a US-only trading day, and vice versa.
6. **New suspicious jumps** (above 50%) are reported with a note whether a split happened around that date, but never corrected automatically: a human decides and adds them to the artefact list.

Shares outstanding and short interest get a `known_at` timestamp: the moment the number was published, not the date it refers to. Every model reads data through a point-in-time accessor (`PointInTime.as_of(t)`), which only returns what was known at time t.

Features (`features.py`)

About 35 features per stock per day, computed on the stock's own trading days with backward-looking windows only:

Group	Features
Momentum and trend	returns over 1, 5 and 21 days; 6-1 and 12-1 month momentum; distance to the 52-week high, the 50- and 200-day average; drawdown from the all-time high
Volatility and risk	volatility over 20 and 60 days; Parkinson volatility from daily high/low; largest daily return in a month; skewness; beta to the stock's own market (STOXX 600 or S&P 500) and to bitcoin; idiosyncratic volatility
Volume and liquidity	abnormal volume; Amihud illiquidity; share of days without trades
Earnings	last surprise, days since and until results, average past price jump on results
Size and short interest	market cap and short interest as known at decision time
Distress	a 0–3 score: far below the 52-week high, volatility above 100%, 12-month momentum below −60%
Market	VIX level and change, 10-year US rate and change, month, days to year end

Timing convention. A feature dated day d uses only data up to the close of d. The decision moment is 08:00 the next morning, when all closes of d are known, including the US close and

EUR/USD. The target the models learn is the log return from the **open** of the next trading day to the close h days later:

$$y_h = \ln\left(\frac{\text{close}_{d+h}}{\text{open}_{d+1}}\right), \quad h \in \{1, 3, 5, 10, 20, 48\}$$

Starting at the next open, not at today's close, matters: an order placed in the evening is executed at the next open, so the overnight gap is not ours to earn.

Leakage tests

Leakage (information from the future sneaking into a feature) is the classic way backtests lie. Each stage has an automated test that tampers with everything published after a date and checks that nothing computed for that date changes, while the target does change (so the test is actually sensitive): `python br.py features test`, `python br.py risk_test` and `python br.py ret_test`. All pass.

4. Risk model: how much will each stock move?

The chosen volatility model, an average of a HAR regression and a gradient-boosting model, blended with option prices for US stocks, beats the industry-standard EWMA at every horizon on 16 months of data it never saw.

Volatility is the most important input of the whole system. It decides which stocks can produce the extreme outcomes a top-3 finish needs, and unlike returns, it is genuinely forecastable: calm stocks stay fairly calm, and turbulent periods cluster.

What is forecast

For every stock and every decision day d, the model forecasts the **realised variance** over the next h of the stock's own trading days, for six horizons:

$$RV_h(d) = r_{d+1}^2 + r_{d+2}^2 + \cdots + r_{d+h}^2, \quad h \in \{1, 3, 5, 10, 20, 48\}$$

with r the daily log return in euro. The 48-day horizon is the whole competition; the shorter ones feed the scenario generator, which needs the shape of risk over time, not just its total.

Candidate models

Six families were built, each walk-forward (refitted on past data only) so that every historical forecast is one the model could really have made at that time:

Model	Idea	Strength	Weakness
Rolling 60 days	average squared return over the last 60 days	simple, robust	slow, ignores everything else
EWMA ($\lambda = 0.94$, 0.97, 0.99)	exponentially weighted: recent days count more (RiskMetrics)	industry benchmark	assumes today's level lasts forever

Model	Idea	Strength	Weakness
GARCH(1,1) per stock	variance mean-reverts to a long-run level V: forecasts for long horizons pull towards V	the textbook model	fitted per stock, noisy; refitted each quarter on 1,000 days
HAR, pooled	regression of log future variance on log past variance over 1, 5, 22 and 66 days, one model for all stocks, plus blocks of extra information	captures that volatility has short and long memory; pooling makes it stable	linear
Gradient boosting (LightGBM)	trees on the same inputs; as a correction on HAR, or on its own	finds non-linear effects and interactions	cannot extrapolate beyond the training range
Option-implied volatility	the market's own forecast, read from option prices	forward-looking, includes known events like earnings dates	only for US stocks; biased upwards (a risk premium)

The three classical models, with r_t the daily log return and σ_t^2 the forecast daily variance. EWMA, a weighted average of past squared returns:

$$\sigma_{t+1}^2 = \lambda\,\sigma_t^2 + (1-\lambda)\,r_t^2$$

GARCH(1,1), which adds a pull towards a long-run level:

$$\sigma_{t+1}^2 = \omega + \alpha\,r_t^2 + \beta\,\sigma_t^2, \qquad \bar{\sigma}^2 = \frac{\omega}{1-\alpha-\beta}$$

HAR, one regression for all stocks at once, on realised variance over a day, a week, a month and a quarter (c is a small constant that keeps the logarithm finite, x the optional feature blocks):

$$\ln\!\left(\frac{RV_h}{h} + c\right) = b_0 + b_1 \ln RV^{(1)} + b_2 \ln RV^{(5)} + b_3 \ln RV^{(22)} + b_4 \ln RV^{(66)} + \gamma^\top x + \varepsilon,$$

$$RV^{(k)} = \frac{1}{k}\sum_{j=0}^{k-1} r_{t-j}^2$$

The HAR model adds information in fixed blocks, each kept only if it improved the forecast in the selection period: price range (Parkinson variance over 5 and 22 days), the market (VIX and its change), earnings inside the horizon (with the stock's typical earnings jump), asymmetry (negative returns), size and liquidity, short interest, and distress signals. For EWMA, GARCH and rolling models, which have no features, a separate earnings module adds the expected jump when results fall inside the horizon.

How models were judged

Every model was scored on exactly the same rows (date × stock where every model has a forecast and the outcome is known), with:

- **QLIKE**, the standard loss for variance forecasts (formula below). It punishes underestimating risk more than overestimating it, which is the right asymmetry here.
- **Diebold–Mariano t-statistic** against EWMA 0.94, with Newey–West standard errors because overlapping horizons make errors autocorrelated. Below -2 means convincingly better.
- **Bias**: mean realised variance over mean forecast; 1 is right on average.

With RV the realised variance and F the forecast, and d_t the daily loss difference between a model and EWMA:

$$\mathrm{QLIKE}(RV, F) = \frac{RV}{F} - \ln\frac{RV}{F} - 1,$$

$$DM = \frac{\bar{d}}{\widehat{\mathrm{se}}_{NW}(\bar{d})}, \qquad d_t = \mathrm{QLIKE}_t^{\mathrm{model}} - \mathrm{QLIKE}_t^{\mathrm{EWMA}}$$

The choice

The **average of HAR and boosting-on-its-own** won in the selection period at 1–10 days, and after the check period it was also adopted for 20 and 48 days. Averaging two good but different models is a classic, robust gain: their errors partly cancel. The forecasts are converted back from logs with a smearing correction (Duan), because the exponential of an average log is lower than the average level.

$$\hat{v}_h = h\left(e^{\hat{y}} \cdot \frac{1}{n}\sum_{i=1}^{n} e^{\hat{\varepsilon}_i} - c\right)$$

For US stocks with usable options, the forecast is blended with the option-implied variance:

$$v_h = \lambda\,(c \cdot IV)^2\,\frac{h}{252} + (1-\lambda)\,v_h^{\mathrm{base}}, \qquad c = 0.9,\ \lambda = 0.7$$

c corrects the volatility risk premium (options are systematically priced a bit too high), and λ sets how much we trust the options. Historical option data per stock is not freely available, so c and λ were tested where history exists: the VIX against the S&P 500 and the VXN against the Nasdaq 100, 2002–2021, which includes the dot-com crash, 2008, 2011, 2018 and Covid. A pre-registered rule said: if the estimates on 2002–2010 and 2011–2021 agree with the full estimate, use it; otherwise use the literature values $c = 0.9$ and $\lambda = 0.7$. The implied volatility per stock is taken around the money (strikes at 90–110% of the price), interpolated in total variance between expiries, and converted from dollar to euro. A quality filter (at least four quotes near the price, a median bid-ask spread below 30% of the mid price) drops unreliable chains; on a normal weekday about 145 of the 150 US stocks pass.

Correlations

Volatility per stock is half of risk; the other half is how stocks move together. Correlations are always estimated on **two-day returns**, because Europe and the US close at different hours: a US move after the European close shows up in European prices the next day. Three estimators

were compared on random 10-stock portfolios in 2019–2021, each judged only on the correlation part (the realised volatility of each stock was plugged in):

- the sample correlation over 500 trading days (too noisy for 369 stocks),
- Ledoit–Wolf shrinkage towards the average correlation,
- **a statistical factor model**: the 10 strongest principal components plus a stock-specific part (`fac_10`).

The factor model won. It captures the big common movements (the market, Europe versus the US, sectors and themes) and treats the rest as independent noise, which is what a portfolio needs. Recent listings without 500 days of history (Cerebras, SpaceX, Quantinuum, Circle and others) get their factor loadings from a regression on the factor returns over the days they do have, shrunk towards the average with weight $n/(n+60)$, where n is the number of two-day observations. So a stock with little history is treated as average until the data says otherwise.

$$\tilde{\beta}_i = \frac{n}{n+60}\,\hat{\beta}_i + \frac{60}{n+60}\,\bar{\beta}$$

Themes (clusters of stocks that move together) were built by hierarchical clustering after removing the strongest common factors, but they turned out unstable from quarter to quarter (adjusted Rand index about 0.24) and are used for information only, not as a model input.

5. Return model: a small, honest edge

The return model ranks stocks slightly better than chance (a rank correlation of 0.03–0.08), which is statistically real over seven years but small: it tilts the expected return by about ±1% over the competition, against swings of tens of percent.

Returns are notoriously hard to predict, and an overconfident return model is the fastest way to ruin a strategy. So the design goal was modest: forecast only what can be forecast, measure it honestly, and shrink it towards zero in proportion to the evidence.

What is forecast

Only the **stock-specific** part of the return: how much better or worse a stock does than the average stock on the same day. The market as a whole is not forecast; every stock gets the same market drift of 7% per year (about +1.3% over 48 trading days). Forecasting the market is a different and harder game, and for ranking players against each other the common market move largely cancels out.

Six candidates

Model	What it is	Parameters
zero	every stock equal (forecast 0): the benchmark to beat	none
CAPM	(beta − average beta) × 7% × h/252: high-beta stocks earn more when the market rises	none

Model	What it is	Parameters
composite	average of four signals with signs fixed from the academic literature: 12-month momentum (+), size (−), illiquidity (+), last earnings surprise (+)	1 scale
elastic net	regularised linear regression on all features plus missing-value indicators (ridge, lasso or a mix, chosen at each refit)	~45 coefficients
gradient boosting	LightGBM trees on the same features	trees
neural network	feed-forward 32-16-8 with ReLU, PyTorch on the GPU, weight decay chosen at each refit, early stopping, average of 3 networks	weights

Features are converted each day to ranks between −0.5 and +0.5 within that day's cross-section, which makes them robust to outliers and comparable over time without using any future information. Market-wide features (VIX, rates, month) are the same for every stock on a given day, so they say nothing about the ranking and are left out.

Walk-forward with embargo

Each model is refitted every quarter (boosting every half year) on data up to the refit date **minus h + 5 trading days**. That embargo matters: a row from yesterday has a 48-day target that is not finished yet, and training on it would leak the future. Hyperparameters (penalty, number of leaves, weight decay) are chosen on the last 20% of the training dates, with a gap of h days, never on the period being forecast.

Rank IC, not R^2

The main metric is the **rank information coefficient**: each day, the correlation between the predicted ranking and the realised ranking of stocks, averaged over days, with a Newey–West t-statistic because overlapping horizons make consecutive days dependent. R^2 is reported too, but it is misleading for this purpose: an R^2 of 0.3% sounds useless, yet a consistent IC of 0.05 across 370 stocks every day is a real, exploitable ordering. What the strategy needs is the ordering, not a point forecast.

$$IC_t = \rho_{\text{Spearman}}(\hat{f}_{\cdot,t},\ y_{\cdot,t}), \qquad \overline{IC} = \frac{1}{T}\sum_{t=1}^{T} IC_t, \qquad t_{IC} = \frac{\overline{IC}}{\widehat{\text{se}}_{NW}(\overline{IC})}$$

Two more checks keep the model honest: the IC on only the larger half of the stocks (a guard against survivorship bias, since delisted small caps are missing from free data), and the **top-minus-bottom** spread, the realised return of the best 10% of forecasts minus the worst 10%.

Choosing and shrinking

The selection rule was fixed before looking at results: per horizon, take the two models with the highest IC in 2019–2021; if their difference is not significant, average their daily z-scores, otherwise take the best. The chosen blends:

Horizon	Model
1 and 3 days	average of neural network and elastic net
5 days	average of boosting and neural network
10 days	average of CAPM and neural network
20 and 48 days	average of CAPM and boosting

The forecast that enters the scenarios is

$$\mu_h = 7\% \cdot \frac{h}{252} + w \cdot b \cdot \hat{f}_h, \qquad w = \max\left(0,\ 1 - \frac{1}{t_{IC}^2}\right)$$

where b is the calibration slope (how many percent of realised return one unit of forecast is worth) and w is a confidence weight that goes to zero as the evidence weakens. Any horizon with $t < 2$ or a non-positive IC gets $w = 0$: no tilt at all. After the final test, w and b were measured once more over all data from 2019 to now (the pre-agreed refit): every horizon kept $t \geq 2.95$, so w is between 0.89 and 0.99, with b between 0.66 and 1.52.

Why it is used even though it is small

The 90th-minus-10th percentile spread of the stock-specific forecast is about 2.3% over 48 days. A single volatile stock moves ±30–50% in the same period. So the return model does not pick the portfolio; it breaks ties. Between two stocks with a similar risk profile it prefers the one with better prospects, and it stops the simulation from assuming that a struggling company will do as well as a growing one. Trading on the signal alone does not pay after costs at short horizons (the script reports the break-even IC, about 0.22 at 1 day and 0.03 at 48 days), which is exactly why the trading rule never acts on small forecast changes.

6. Scenario generator: 20,000 possible competitions

The generator turns the forecasts into 20,000 correlated day-by-day futures for all 369 securities, and its shape was chosen and tested on history: on unseen data its tail percentages deviate 3 times less from reality than a normal distribution's.

Forecasts of mean and variance are not enough for this problem. What decides the competition is the **joint** distribution: which stock ends up best, how far apart the winners and losers spread, and what happens to a whole portfolio when a theme crashes. Monte Carlo simulation gives all of that at once.

Building one future

For every stock i and every trading day t until 22 January, the generator draws a daily log return

$$r_{it} = m_{it} + \sqrt{v_{it}}\, z_{it}, \qquad z_t \sim N(0, R)$$

- **Drift and variance per day.** The models forecast cumulative mean and variance at six horizons (1, 3, 5, 10, 20, 48 days). The generator interpolates those cumulative curves between the horizons and takes the day-by-day differences. Every horizon in the simulation therefore has exactly the forecast variance and mean, including the term structure (for example, turbulent now and calmer later).
- **Correlation.** Independent standard normal shocks are multiplied by a square root L of the correlation matrix ($LL^\top = R$), which gives them exactly the estimated co-movement. If the estimated matrix is not a valid correlation matrix (it happens with estimated matrices), it is first repaired to the nearest valid one.
- **Holidays.** US stocks do not move on Thanksgiving (26 November) and Martin Luther King Day (18 January); their variance is spread over their own 46 trading days instead of 48.
- **Data gaps.** When the latest prices are a few days before the start, those days are simulated as well and then dropped, so every future starts from the correct uncertainty at the opening bell.

Adding up the 48 daily returns gives one path for every stock to the final day.

Fat tails without a fat-tailed distribution

Real returns have fatter tails than a normal distribution: more extreme days, more extreme 48-day outcomes. The generator can add them in two ways:

- **Stress days**: a random scale per day shared by all stocks (a multivariate Student-t with 5 degrees of freedom): occasional crash days on which everything moves at once.
- **Volatility uncertainty**: each future draws a random multiplier on the variance, one for the whole market and one per stock, lognormal with mean 1 (so the expected variance is unchanged). Some futures are calm, some are turbulent, and some single stocks go wild. Mixing normal distributions with different variances produces fat tails over 48 days, which is the horizon that matters.

In formulas, with g and g_i standard normal draws per future, s_M and s_S the two uncertainty levels, and ν the degrees of freedom of the stress days:

$$v_{it} \;\to\; v_{it}\,\exp\!\big(s_M\,g - \tfrac{1}{2}s_M^2\big)\exp\!\big(s_S\,g_i - \tfrac{1}{2}s_S^2\big)\,, \qquad z_t \;\to\; z_t\sqrt{\frac{\nu-2}{\chi^2_\nu}}$$

Which combination fits reality was not assumed but **tested**. For 40 shapes (normal or Student-t, market uncertainty 0 to 0.75, stock uncertainty 0 to 1.0), the generator was rebuilt on every month-start in 2019–2021 using only what was known then, and compared with what actually happened. The test is the **probability integral transform**: in which percentile of the simulated outcomes does the real outcome fall? A good simulation puts 1% of real outcomes below its 1st percentile, 5% below its 5th, and so on, in both tails. This was checked for single stocks (with and without the market move), for the 10 most volatile stocks, for random 10-stock portfolios, for the best stock of each period and for the spread between stocks. The score is a chi-square-like sum of squared, standardised deviations, weighted by how many independent observations each test has.

The winner: **normal daily shocks with volatility uncertainty 0.75 for the market and**

0.75 per stock. Its deviation in 2019–2021 was 64, against 420 for plain normal shocks. Stress days did not add enough to be kept (each extension had to improve the score by at least 5%).

A cap on fantasy

With fat tails, a few simulated stocks can rise absurdly. Each path is capped at +1500% (a factor 16) over the competition. That is not arbitrary: the largest real 48-day rise in 2019–2021 was +1201%, and a lower cap (450% was considered) would cut off outcomes that really happen and that a top-3 finish can depend on.

Speed

The generator runs on the GPU through PyTorch when available, with a numpy fallback on the CPU. 20,000 futures for 369 stocks over 48 days take about half a second on an RTX 5080. Scenarios are never stored (they would take 1.4 GB): every script regenerates them from the saved parameters with a fixed seed, so the same seed always gives the same futures.

A self-test (`python br.py scen_core test`) checks that known inputs come back out: the mean and variance at every horizon, the correlation of 20-day returns, independence of days, the holiday handling, the fat tails, and the cap.

7. Game engine: the rules, exactly

The game engine (`game.py`) plays any strategy under the exact competition rules in thousands of futures at once, and 19 automated tests check each rule.

A simulation that ignores the rules flatters every strategy. Costs, the 5-order limit and whole shares sound like details, but together they decide how fast a portfolio can be built or changed, and how much a trade must gain before it is worth doing.

One trading day in the engine

1. **Before the open**, the strategy sees everything known up to the previous close (cash, positions, prices, the history of the path) and submits orders.
2. **At the open**, orders are executed: first all sells (they free up cash), then the buys, in the order the strategy gave. Each order is checked against every rule; a rejected order is dropped entirely (no partial fills) and does not count towards the daily limit.
3. **At the close**, the quiz money and, on 4 December, the Finance Avenue bonus are credited, and the portfolio value of the day is recorded.

The scenarios contain closing prices only, so the opening price of day d is approximated by the close of day $d - 1$. The final value is cash plus positions at the closing prices of 22 January, without selling costs, as in the rules.

What is enforced

Rule	How the engine applies it
Max 5 orders per day	the 6th executed order of a day is rejected; rejected orders do not count
1 buy and 1 sell per security per day	a second buy of the same stock that day is rejected

Rule	How the engine applies it
10% cap at purchase	a buy that would bring the position above 10% of the total value is rejected; a position that grows above 10% through price gains stays (no forced sale), but cannot be topped up
Costs	€5 + 0.35% on every buy and sell
Whole shares	a buy of X euro buys floor(X / price) shares
Cash never negative	a buy larger than the available cash is rejected in full
Quiz and Finance Avenue	credited at the close, usable from the next open; no quiz assumed on 24 and 31 December (the rules don't say which two days have none)

The engine is vectorised over futures: it plays one strategy in 6, 500 or 6,000 futures in a single pass of numpy array operations, and a test checks that 6 futures at once give exactly the same result as 6 futures one by one.

Strategies as policies

A strategy is a small object that receives the state before each open and returns orders. The main one is **buy-and-hold with top-ups**: buy the chosen 10 stocks over the first two days (5 orders per day, €4,750 each), then every 5 days use the accumulated quiz money to top up the position that has fallen furthest below its target weight (9.5%), as long as at least €500 can be invested and no position goes above 10% at purchase. The 9.5% target instead of 10% leaves room for price moves between Monday and Tuesday, so that no Tuesday order is rejected by the cap.

The tests read like the rulebook: a €4,900 buy costs exactly €22.15; 10 buys on day 1 execute only 5; an 11% buy is refused; selling before buying on the same day lets the proceeds fund the buy; with no trades and every quiz answer correct the final value is exactly €55,100 (€50,000 + 46 × €100 + €500).

8. The competitor field: who are we playing against?

The field model simulates 32,000 players of five types in six differently composed fields, and a strategy is only chosen if it does well across all six, because nobody knows exactly who will play.

The prize depends on our rank, so we need the competitors' outcomes in exactly the same futures as ours. A player who holds Nvidia gains when Nvidia gains, in the same scenario. That shared exposure is what makes this more than a forecasting problem.

Five player types

Type	Portfolio	Quiz (chance per question)
Inactive	stays in cash, or 1–2 positions	30%; but 40% of inactive players answer faithfully, so cash + quiz gives them about +9%
Cautious	3–8 ETFs and large well-known stocks	70%

Type	Portfolio	Quiz (chance per question)
Casual	5–12 popular names: large caps, Belgian stocks chosen 2.5× more often	60%
Momentum	8–12 of the 40 biggest risers of the last 6 months	90%
Gambler	10 of the 50 most volatile stocks, often concentrated in one theme	90%

Players hold equal positions of 9.5%, pay the same costs, and a small share attends Finance Avenue (2% of inactive players up to 15% of momentum players and gamblers). Each player gets an individual quiz skill around the type's average, so the field contains a realistic spread of quiz income.

Portfolios are drawn with weighted sampling without replacement (the Gumbel top-k trick, which draws thousands of portfolios in one vectorised step). The inactive players who answer the quiz faithfully turned out to matter: without them, a strategy of doing nothing and only answering the quiz scored a 3.8% chance of a student prize in an early version, which is clearly wrong. With them, a pure quiz strategy correctly scores nothing.

Six fields

The base field has 20% inactive, 20% cautious, 30% casual, 15% momentum and 15% gamblers among the general players, and more gamblers (25%) among the 15% students. The five variants shift that mix:

Field	What changes
basis	the mix above
minder actief (less active)	more inactive and cautious players, fewer gamblers
meer actief (more active)	more momentum players and gamblers, slightly better quiz scores
veel gokkers (many gamblers)	gamblers up to 30% of general players and 40% of students
een hot thema (one hot theme)	80% of gamblers concentrate in the single theme with the strongest momentum
momentumjagers (momentum chasers)	momentum players up to 30–35%

Every block of futures gets a **freshly drawn field**, so a strategy cannot become tuned to one accidental set of competitors.

Prize allocation

In each future, the engine ranks our final value among all players. Prizes are handed out from the highest amount down (G1 €10,000, G2 €2,500, S1 €2,500, G3 €1,500, S2 €1,500, S3 €1,000), and each person gets only their highest prize: a student who finishes second overall takes G2, and the student prize passes to the next student. For speed, only the top 8 values per ranking are kept in each future, which is enough to resolve every pass-down.

What it does not model

Competitors buy and hold; real players trade. The justification: players who chase winners do not end higher on average, and the spread of their outcomes is approximated by the gamblers. During the competition this limitation shrinks every week, because the weekly routine replaces the simulated leaders by the **real** portfolios of the top players (section 10). The trial round supports the field's central assumption: its leaders were almost entirely invested in AI and chip stocks, and the top students held few, concentrated positions with a lot of cash.

9. The starting portfolio: search on one set, judge on another

The starting portfolio is found by a parallel local search over 20 starting points on one set of 6,000 futures and judged on a fresh set of 6,000, which removes the luck of the search from the reported result.

The search problem

Choosing 10 stocks out of 349 candidates gives about 7×10^{18} combinations, so trying them all is impossible. The objective (expected prize money) has no gradient and many local optima. A simple, robust method works well here: **local search**. Start from a portfolio; try replacing each of the 10 stocks by each candidate; keep a swap only if it raises the expected prize by at least 1%; repeat until no swap helps (at most 12 rounds).

During the search, each portfolio is scored with a fast approximation of buy-and-hold on the **search set** (6,000 futures, each with a freshly drawn field): equal positions, costs, quiz money, our value compared with the stored top-8 values of each field. That takes milliseconds per portfolio, and the search evaluates hundreds of thousands of them.

Twenty starting points, in parallel

Local search ends in the nearest local optimum, so where it starts matters. The four "sensible" starting points (the most volatile stocks, the best forecasts, the hot theme, volatile stocks with good forecasts) turned out to all get stuck in the same mediocre optimum. Random starting points did much better. The final version uses 4 fixed starts, the best of 300 random portfolios, and 15 more random starts, all searched at the same time in separate processes on 16–20 CPU cores. The search set itself is also built in parallel. A run over all candidates takes about 40–60 minutes.

The control set: no winner's curse

After searching through hundreds of thousands of portfolios on the same 6,000 futures, the best one is partly the luckiest one: its score on the search set is optimistic. So the six best **distinct** end portfolios, plus two simple benchmarks, are replayed on a **control set** of 6,000 new futures with new fields, this time with the full game engine (two days of 5 orders, whole shares, top-ups with quiz money, in the exact buying order of the order list). The portfolio with the highest expected prize on the control set is chosen, and its control score is the one reported.

Candidates and exclusions

The candidate pool started as the 70 most volatile stocks plus the 20 best return forecasts plus the hot theme, for speed. When the optimiser started picking stocks right at the edge of that pool, the pool was opened to all 349 affordable stocks (an option `--kandidaten alle`). That turned out to matter a lot (section 12).

A few stocks are excluded by hand, under one written rule: only for a concrete, public, dated fact that the model cannot see and that pushes the price in a predictable direction during the competition, or makes the model's inputs unreliable. Examples are an announced share issue (Sif Holding: an extraordinary shareholder meeting on 22 December to authorise one, after doubts about its going concern) or a rights issue that may be unadjusted in the price history (Avantium). "Bad company" or a bad feeling is explicitly not a reason. Bpost and Alstom were also excluded, as a judgment call; measured on the same control set, that cost nothing (€251 with them available versus €259 without).

Is the search better than luck?

Two checks on the control set answer that:

Portfolio	Expected prize money	P(top 3 students)
chosen (search over all 349 stocks)	€317 ± 20	5.6%
chosen (search over the 70 most volatile)	€259 ± 18	4.7%
the 10 best return forecasts	€141 ± 11	3.4%
the 10 most volatile stocks	€135 ± 12	3.2%

And 2,000 random portfolios drawn from the volatile part of the pool, scored on the same control set with the fast approximation, had a median of €12 and a best of €107; the chosen portfolio scored €272 on the same measure, better than all 2,000. The ± is the standard error of the average over 6,000 futures; because all portfolios are scored on the same futures, the differences between them are much more precise than the ± suggests.

Robust to the type of field?

The expected prize is an average over six field types. A portfolio could win that average by being very strong in one of them. So the two finalists were replayed per field type, on the same futures:

Field	Volatile portfolio (70 candidates)	Contrarian portfolio (all candidates)	Difference
basis	€241	€309	+€68
minder actief	€285	€347	+€62
meer actief	€217	€274	+€57
veel gokkers	€222	€289	+€67
een hot thema	€312	€375	+€63
momentumjagers	€232	€306	+€74

The contrarian portfolio wins in every field type, by a similar margin, so its advantage does not hinge on one assumption about the field.

10. Trading during the competition: act only on confirmed gains

During the competition the portfolio is re-evaluated every Friday from the real standings, and it trades only when an action wins on two independent halves of the simulated futures; in tests that added €25 ± 24 over buy-and-hold with about 0.2 actions per competition.

First attempt: fixed rules (rejected)

The intuitive approach is a rule such as "if you are far behind near the end, take more risk" (catch-up) or "if you are ahead, lock in the lead by selling your wildest positions" (lock-in). Both were implemented (`rules.py`), with their thresholds searched on one set of futures and compared on another, paired against plain buy-and-hold with top-ups, at weekly, daily and end-weighted checkpoints. Neither beat buy-and-hold. Lock-in was even harmful: selling the volatile positions also sells the chance to stay ahead when competitors catch up. The lesson: a fixed rule cannot know whether a specific action helps in a specific situation.

The rollout

Instead of a rule, the decision is computed: from the current state, simulate the rest of the competition for every possible action and pick the one with the highest expected prize. Each Friday after the close:

1. **State**: our positions and cash, every competitor's current value, the days left.
2. **Inner futures**: fresh scenarios for the remaining days only.
3. **Candidate actions**, at most a few hundred: *swap* (sell position i entirely, buy stock j with the proceeds and free cash, up to just below 10%), *sell to cash* (partly locking in), *new buy* (buy j with free cash of at least €1,500). The stocks j come from the most volatile affordable stocks plus the hot theme. Every action pays the real costs.
4. **Expected prize** of doing nothing and of every action, against the competitors in those same inner futures.

The competitors that matter are those who can still reach the top 3: the best 600 general players and the best 600 students at that moment.

The winner's curse, and the fix

The first version acted far too often. Comparing a few hundred noisy estimates, the best one almost always looks better than doing nothing, even when none of them is: in 5 of 10 test situations it traded without a real gain, and it scored €74 below buy-and-hold.

The fix is a statistical split. Half of the inner futures **choose** the best action; the other half, which played no role in the choice, **confirm** it. The action is executed only if it is better than doing nothing on both halves by more than twice the estimation uncertainty. After the fix, the rollout made no spurious trades in the test situations, and a self-test checks both sides: it must take a clearly better swap, and it must leave noise alone. With the split, the gap shrank to −€56, and that remaining gap turned out to be a bug: the rollout skipped the quiz top-ups on checkpoint days. A test now checks that a rollout without actions is identical to buy-and-hold, top-ups included.

The outcome, paired on the same futures: **+€25 ± 24 over buy-and-hold**, with about 0.2 actions per competition. That is a modest, honest improvement: the rollout mostly does nothing,

and acts when the situation clearly calls for it, such as a position that has collapsed and can be swapped for a better chance. A daily rollout was considered and rejected: more checkpoints means more chances to act on noise and more costs, for little gain over nine weeks.

The weekly routine with the real standings

In the live competition the simulated state is replaced by the real one, read from the Beursrally website:

1. **`stand_inlezen.py`** watches the clipboard. On the website the player presses Ctrl+A, Ctrl+C on their own portfolio, on both rankings, and on the portfolios of the top 5 students and top 5 general players. The script recognises each page, parses the Belgian number format, merges multi-page portfolios, checks that positions plus cash add up to each page's total, maps every position to the data by ISIN, and lists which pages are still missing. About 7 minutes a week, no typing. It only reads pages the player opens and copies; there is no scraping and no automated order entry.
2. **`week.py`** runs the rollout from that real state: our real positions and cash; the copied leaders with their **real** positions; and the rest of the field simulated with the player types, with current values that follow the real ranking (ranks 1–20 exactly, interpolated down to our own rank). It prints "do nothing" or the orders for Monday: sells first, then buys, then quiz-money top-ups.

A weekly news check stays manual, under the same exclusion rule as at the start: share issues, takeover bids, suspensions, covenant problems.

11. Validation: choose, check, and test once

Every model was chosen on 2019–2021, checked on 2022 to mid 2025, and tested once on mid 2025 to November 2026: volatility and scenarios held up everywhere, the return model held at 1–5 days and weakened at longer horizons.

The protocol

The most common way to fool yourself with financial models is to try things until the backtest looks good. The protocol here was fixed in advance:

Period	Dates	Role
Selection	2019 – 2021	all choices are made here: models, settings, the scenario shape
Check	2022 – mid 2025	looked at after the choices; modest, pre-stated adjustments allowed (fallback rules)
Final test	mid 2025 – November 2026	looked at **once**, after everything was fixed; no adjustments afterwards

The check period covers a very different market (the 2022 bear market and rate shock, then the AI rally), which makes it a demanding test of whether the 2019–2021 choices generalise. One

adjustment came out of it: the volatility average of HAR and boosting, chosen at 1–10 days, was also adopted at 20 and 48 days. After the final test, one pre-agreed step followed: re-measuring the return model's confidence w and calibration b on all data from 2019 to now, with the same rule.

Volatility

The chosen model (average of HAR and boosting) against the EWMA 0.94 benchmark, in the final test. The option blend is not part of this test, because free historical option data per stock does not exist; it was tested on index data instead (section 4).

Horizon (days)	1	3	5	10	20	48
Diebold–Mariano t vs EWMA	−8.3	−8.4	−8.5	−7.2	−5.5	−3.0
Bias (realised / forecast)	1.06	0.99	0.99	1.01	1.01	1.04

Convincingly better at every horizon (t below −2), and unbiased on average. In the check period the picture was the same (t between −5 and −12, bias about 1). At 20 and 48 days the average also beat HAR alone (QLIKE 0.258 vs 0.279 at 20 days, 0.191 vs 0.207 at 48 days).

Returns

The t-statistic of the rank IC of the chosen model per horizon and period (above 2 = convincing):

Horizon (days)	1	3	5	10	20	48
Selection 2019–2021	6.74	3.68	3.31	3.07	3.10	3.17
Check 2022 – mid 2025	6.08	2.36	2.42	1.79	1.57	1.52
Final test	3.23	2.21	3.40	0.78	1.01	0.74
All of 2019 – now	9.60	4.76	4.78	3.24	3.13	2.95
IC, all of 2019 – now	0.037	0.031	0.035	0.039	0.052	0.077

The honest reading: at 1–5 days the signal is robust. At 10–48 days the IC stayed positive in every period but was not significant on its own in either out-of-sample period, which is consistent with a weaker signal and also with a slow fade. The main culprit is visible in the detailed output: the CAPM part (the bet that high-beta stocks outperform) had an IC near zero in the final test, while the neural network alone stayed strong. Switching to the network after seeing that would be choosing on the test set, so the pre-agreed rule decided instead: pooled over all data, every horizon has $t \geq 2.95$, so all horizons keep w between 0.89 and 0.99. The practical effect of this choice is small, because the tilt is small (section 5).

Scenarios

The deviation score of the tail percentages (lower is better) for the chosen shape against plain normal shocks:

Period	Chosen shape	Normal shocks
Selection 2019–2021	64	420
Check 2022 – mid 2025	135	315
Final test	26	76

In the final test, single stocks land in the simulated percentiles almost exactly as they should (3.8–5.1% below the 5th percentile, 0.3–0.6% below the 1st). The simulated return per volatility group matches reality (most volatile fifth: +3.36% simulated, +3.32% real), so the model does not systematically favour risky stocks. The best stock after 48 days, the outcome a top-3 finish depends on, is of the right size: median +87% simulated against +108% real, 90th percentile +240% against +209%.

One weakness showed up in the check period and is kept in mind: the simulation underestimated how hard a basket of the most volatile stocks crashed together in 2022.

12. Results: the model bets against the crowd

With all stocks available, the optimiser chose a contrarian portfolio of "AI losers", luxury and consumer stocks and a few volatile single names, because those tend to rise exactly in the futures where the AI-heavy field falls.

Two very different answers

With the 70 most volatile stocks as candidates, the best portfolio was what intuition expects: Lucid, Moderna, Genfit, Ontex, Snap, Onward Medical, Teleperformance, Ubisoft, lululemon and Air France-KLM. Volatile names with their own stories, mostly unrelated to AI. Expected prize: €259 ± 18.

With all 349 stocks as candidates, the answer changed: Moderna, Ontex, Teleperformance, lululemon, Air France-KLM, Kering, Randstad, EssilorLuxottica, Rémy Cointreau and Pernod Ricard (data up to 9 November 2026). Several of these are not volatile at all (Pernod Ricard about 32% a year). The other finalists of the same run share the theme: Capgemini, Accenture, Adobe, Autodesk and Wolters Kluwer, companies the market fears will be hurt by AI. Expected prize: €317 ± 20, on the same futures.

Why a calmer portfolio wins more

It looks paradoxical: a top-3 finish needs an extreme outcome, yet the model picked lower-volatility stocks. The answer is the competition, not the stocks.

In futures where the AI trade keeps booming, thousands of players rise together and the top 3 goes to whoever had the most luck among them. A portfolio outside that theme almost never wins there, whatever it holds. In futures where the AI trade cracks, the crowd falls together and the bar for the top 3 drops sharply. Those are the futures where a non-AI portfolio can win, and the model maximises its strength exactly there. Stocks that tend to move **opposite** to

AI winners (the "AI losers") are the most valuable in those futures, because their rise coincides with the crowd's fall. The model never had to be told about AI: it found this structure in the correlation matrix and the simulated field.

The core is stable across runs: Moderna and Teleperformance appeared in all twelve finalists of both runs, Ontex and lululemon in ten of them. The last few names vary from run to run, which shows they hardly matter.

What the bet depends on

The advantage held in all six field types (section 9), so it does not rest on one assumption about how active or how concentrated the field is. What it does rest on is the market structure of the last two years: that AI winners and AI losers keep moving in opposite directions for nine more weeks. If that pattern fades, the contrarian stocks become ordinary low-volatility stocks with little chance of a top 3. The weekly rollout limits that risk: with the real leaders' portfolios in the model, it can swap if the situation clearly calls for it.

The final starting portfolio is recomputed on Sunday 15 November with Friday's data, so the names may still shift; the type of portfolio is expected to stay the same.

13. Limitations, and what was left out on purpose

The system's main blind spots are news, the real behaviour of competitors, and how long today's market structure lasts; each has a mitigation, and several tempting additions were deliberately not built.

Known limitations

- **The model does not read news.** Share issues, takeover bids, suspensions and covenant problems are invisible to price-based models but move prices in a predictable direction. Mitigation: a written exclusion rule and a weekly manual news check.
- **Competitors are simulated as buy-and-hold.** Real players trade. Mitigation: the gamblers approximate the spread of active players' outcomes, and from week one the real leaders' portfolios replace the simulated ones.
- **The field composition is unknown.** Mitigation: six different fields, and a portfolio that wins in all of them.
- **Correlations are estimated on the last two years.** The contrarian result depends on the AI-winner versus AI-loser structure persisting for nine weeks.
- **Crash correlation is underestimated.** In 2022 the most volatile stocks fell together harder than the simulation expected.
- **Survivorship bias.** Free data misses companies that were delisted; the return model's IC is also checked on the larger half of stocks to limit the damage.
- **Openings are approximated by the previous close** in the game engine, since the scenarios contain closing prices only.
- **The long-horizon return signal** was not significant on its own out of sample; its effect on decisions is small by design.

Deliberately left out

Idea	Why not
Funds (69 in the universe)	far too calm to contribute to a top-3 finish, and their costs are hidden in their returns
Fundamentals (ROE, cash flow, growth)	slow signals, and free sources are not point-in-time: using today's fundamentals for a 2020 backtest would leak the future
News and sentiment analysis	no free history to validate it, and scraping the competition platform is not allowed
A model of conditional skewness (P(return > threshold) per stock)	the scenarios already give tail probabilities from mean, volatility and shape; a separate classifier would mostly relearn those
An LSTM return model	would need a clean, unseen test period, and the final test was already used; LSTMs rarely beat tuned boosting on cross-sectional ranking; and returns barely drive the decision
Fixed trading rules (catch-up, lock-in)	tested: no gain over buy-and-hold; lock-in was harmful
Daily rollout	more chances to act on noise, more costs, little gain over nine weeks
A second account	not allowed by the competition rules

The common thread: anything added had to be testable out of sample, and had to be able to change a decision. Ideas that failed either test were not built, however sophisticated they sounded.

14. Software, packages and outputs

The whole system is plain Python: 33 modules, one entry point, and files on disk between every stage. There is no database and no server. Each stage reads the previous stage's files and writes its own, so any stage can be rerun, inspected or tested on its own.

Packages, and what each one does here

Package	Used for	Where
yfinance	daily prices, dividends and splits, earnings dates, share counts, option chains	data, risk_iv
requests	FINRA's public short-interest data (US stocks)	data
pandas	every table: prices, features, forecasts, portfolios, rankings	everywhere
numpy	the game engine, local search, rollout and all fast array maths	everywhere
pyarrow	the Parquet file format, which pandas uses to store tables compactly and with types intact	everywhere
scipy	maximum-likelihood fitting of GARCH (`optimize`, `signal.lfilter`); clustering of the correlation matrix into themes	risk_garch, risk_corr

Package	Used for	Where
lightgbm	gradient-boosted trees: the boosted volatility model and the boosted return model	risk_boost, ret_models
scikit-learn	the cross-validated elastic net (`ElasticNetCV`)	ret_models
torch (PyTorch)	the small neural return model, and generating the Monte Carlo scenarios on the GPU	ret_models, scen_core
matplotlib	diagnostic charts of the data	check_data

Two choices are worth defending.

Why a hand-written GARCH instead of the `arch` package. `arch` is the standard library for GARCH models, but it did not install on Python 3.15 under Windows. GARCH(1,1) is a short likelihood function, so it was written directly on top of `scipy.optimize`, with `scipy.signal.lfilter` doing the variance recursion fast. It is also tested against simulated data with known parameters.

What PyTorch actually does. PyTorch is a library for arrays ("tensors") that can live on a graphics card, with automatic differentiation on top. It is used twice. First, for the neural return model: a small network (32-16-8 neurons) whose weights are fitted by gradient descent, three times with different random starts, after which the three forecasts are averaged. Second, and more importantly, for speed in the scenarios: drawing 20,000 correlated paths for about 370 stocks over 48 trading days is millions of random numbers and large matrix products, which the GPU does in about half a second. That speed is what makes the strategy search affordable. Nothing in the scenario code needs a neural network; PyTorch is simply a convenient GPU array library. Without a GPU, the same code runs on the CPU, only slower.

Project layout

```
br.py                 single entry point: python br.py <command>
config/               universe.csv (the 445 products), artefacten.csv
beursrally/
  data/               download, clean, features, data checks
  risk/               volatility models, options, correlations, comparison, build
  returns/            return models, comparison, build
  scenarios/          scenario generator, build, backtest
  game/               game engine and competitor field
  strategy/           start portfolio search, field comparison, rules, rollout
  weekly/             reading pasted pages, weekly decision
  tests/              leakage and correctness tests
docs/notes/           design notes per stage
data/                 everything generated (never in git)
```

`br.py` is a small dispatcher. `python br.py` lists every command; `python br.py risk_build` imports `beursrally.risk.risk_build` and runs it, passing the remaining arguments along. This gives one place to run things from, regardless of where the modules live.

The `data/` folder is in `.gitignore` and has never been committed: besides being large and fully reproducible, it contains other players' names and portfolios copied from the competition pages.

What the system produces

File	Made by	Contents
`data/raw/`, `data/clean/`	data, clean	downloaded and cleaned inputs per source
`data/processed/close_eur.parquet`	clean	daily closing prices in euro, one column per product
`data/features/features.parquet`	features	35 point-in-time features and the targets per stock per day
`data/risk/vol_vandaag.parquet`	risk_build	today's volatility forecast per stock for each of the six horizons
`data/risk/corr_vandaag.parquet`	risk_build	the correlation matrix
`data/risk/themas_vandaag.parquet`	risk_build	which stocks move together (themes from clustering)
`data/returns/rendement_vandaag.parquet`	ret_build	expected return per stock and horizon (`mu_h`)
`data/scenarios/params_vandaag.npz`	scen_build	everything the scenario generator needs, in one file
`data/strategy/orders_start*.csv`	strategy	the start portfolio as orders: product, number of shares, amount
`data/strategy/vergelijking*.csv`	strategy	the finalists compared on the control set
`data/stand/<date>/raw/`	stand_inlezen	the pasted pages, as text
`data/stand/<date>/posities.csv`, `klassement.csv`, `portefeuilles.json`	stand_inlezen	the parsed own portfolio, rankings and leaders' portfolios
`data/stand/<date>/orders_maandag.csv`	week	this week's orders, if any, plus top-ups

The screen output matters as much as the files. `ret_compare` prints, per horizon and model, the rank IC with its Newey–West t-value and the resulting weight. `risk_compare` prints QLIKE, bias and the Diebold–Mariano test against EWMA. `strategy` prints, for each finalist, its expected prize money with a standard error, its chance of a top-3 place in the student ranking, and how it compares to the random benchmark. `week` prints the current expected prize money and whether any action beats doing nothing by more than twice its uncertainty.

Appendix A. Script reference

Every script is run as `python br.py <name>`. They are listed in the order the pipeline runs them. "Research" scripts were used to choose models and settings; "production" scripts run before the start and every week. Many scripts also accept `test`, which runs a self-test with a known answer.

Data

Script	Role	What it does
`data`	production	Downloads prices, corporate actions, earnings dates, share counts and options from Yahoo Finance, and short interest from FINRA. Raw downloads are kept unchanged; `data update` only fetches what is new. `--synthetic` uses `fake_yf` instead of the internet.
`fake_yf`	test	A fake yfinance with reproducible random data and deliberately planted problems, so the data checks can be tested offline.
`clean`	production	Removes incomplete days, converts to euro, and gives point-in-time access to every input. `clean test` checks that changing the future does not change the past.
`features`	production	Builds the 35 features and the six return targets per stock per day, each stamped with when it was known. `features test` is the leakage test.
`check_data`	research	Prints a plain-language check for a few stocks: last price, currency, gaps, and an optional chart.

Risk

Script	Role	What it does
`risk_common`	library	Shared dates, targets, QLIKE loss and rules, so every volatility model is judged identically.
`risk_simple`	research	Rolling 60-day variance and EWMA (0.94, 0.97, 0.99): the baselines.
`risk_garch`	research	GARCH(1,1) per stock, fitted by maximum likelihood with scipy, refitted each quarter.
`risk_earn`	research	Adds an earnings-day jump to the models that have no features of their own.
`risk_har`	research	HAR: one pooled regression of future variance on past variance over 1, 5, 22 and 66 days.

Script	Role	What it does
`risk_boost`	research	LightGBM, either as a correction on HAR or on its own (`--alleen`).
`risk_iv`	research	Implied volatility from US options, blended with the base forecast; backtested at index level.
`risk_forward`	research	Forward test of the option blend weight on forecasts saved since the build started.
`risk_corr`	research	Compares correlation estimators, clusters stocks into themes.
`risk_compare`	research	Scores all volatility models: QLIKE, bias, Diebold–Mariano against EWMA, per horizon and period.
`risk_build`	production	Today's volatility per stock and horizon (HAR + boosting average, option blend for US stocks), the correlation matrix and the themes.

Returns

Script	Role	What it does
`ret_models`	research	Walk-forward forecasts for every candidate model and horizon, with an embargo of h+5 days.
`ret_compare`	research	Rank IC and Newey–West t per model and horizon, picks the model and sets the weight `w`. `--controle`, `--final` and `--alles` select the period.
`ret_build`	production	Refits the chosen models on all data and writes today's expected return per stock and horizon.

Scenarios

Script	Role	What it does
`scen_core`	library	The scenario generator: correlated daily paths on the GPU, with the chosen shape for fat tails.
`scen_build`	production	Packs today's forecasts into the generator's inputs and checks them. `--start` for a later restart date.

Script	Role	What it does
`scen_backtest`	research	Checks that simulated outcomes match what really happened (probability integral transform), and chooses the shape from a grid.

Game, strategy and weekly routine

Script	Role	What it does
`game`	library	The game engine: plays any strategy under the competition rules in thousands of scenarios at once.
`field`	library	The simulated competitors and the prize rules; prints what it takes to reach the top 3 in each field.
`strategy`	production	Searches for the start portfolio with the highest expected prize money; writes the order list. `--kandidaten`, `--uitsluiten`, `--snel`.
`vergelijk_velden`	research	Compares order lists field by field on the same control scenarios.
`rules`	research	Tests fixed trading rules (catch up when behind, lock in when ahead). Result: no gain.
`rollout`	research / library	The weekly re-optimisation from the current state, and its backtest against buy-and-hold.
`stand_inlezen`	production	Reads pasted competition pages from the clipboard and turns them into the own portfolio, rankings and leaders' portfolios.
`week`	production	The weekly decision from the real standings: expected prize money now, and whether any action is worth taking.

Tests

Script	What it checks
`risk_test`	Changing returns after date D must not change any volatility forecast or theme for dates up to D.
`ret_test`	The same for expected returns; `diagnose` locates a leak step by step if one appears.

Appendix B. Questions you can expect, with short answers

What is this project, in one sentence? A system that chooses a portfolio for a nine-week stock-picking competition by maximising expected prize money, using forecasts of risk and return, a Monte Carlo simulation of the market and of the other players, and a search over portfolios.

Why maximise expected prize money and not return or Sharpe ratio? Because only the top three win anything. A portfolio with a good Sharpe ratio finishes comfortably in the top 10% and wins nothing. The objective has to reward the right tail and the ranking, not average performance.

What is Monte Carlo here, concretely? Drawing 20,000 possible futures of the whole market, day by day for nine weeks, consistent with the forecast volatility, correlations and fat tails. In each future, the competition is played with our portfolio and 32,000 simulated players, and the prize is recorded. The average prize over all futures is the expected prize money of that portfolio. It replaces a formula that does not exist.

What does PyTorch do? Two things. It runs the small neural network that is one of the return models. And it generates the scenarios on the graphics card: 20,000 paths in about half a second. That speed is what allows thousands of candidate portfolios to be evaluated.

How do you know there is no look-ahead bias? Every input carries the date it became known, and the decision moment is 08:00 the next morning. Automatic tests change all data after a date D and verify that no feature or forecast for dates up to D changes. Model training uses walk-forward windows with an embargo of h+5 days between training and test data.

Why rank IC rather than R^2? The decision is which stocks to hold, so what matters is the ordering of stocks, not exact return levels. Rank IC measures the ordering and is robust to outliers. R^2 for daily stock returns is close to zero even for a useful model.

An IC of 0.03 to 0.08 sounds tiny. Is it useful? It is typical for return forecasts that hold up out of sample. It is used carefully: the forecast is shrunk with the weight $w = \max(0,\, 1-1/t^2)$, set to zero when $t < 2$, and added on top of a market drift of 7% a year. Returns nudge the choice; risk and the ranking drive it.

Why so many models? Isn't that overfitting? The models were chosen on 2019–2021, checked on 2022 to mid-2025, and the final period was looked at once. Averages of two models are preferred over single winners because they are more stable. The test is not "which model was best" but "does the chosen one hold up on data it never saw".

What is the winner's curse and how did you handle it? When you search over many portfolios on the same scenarios, the best one looks better than it is, partly by luck. The search uses one set of 6,000 scenarios, the finalists are compared on a separate set of 6,000, and every result is compared to random portfolios scored the same way.

Why is the portfolio contrarian? It is not a market view. The leaders all hold similar high-volatility AI and tech stocks. A portfolio that resembles theirs needs to beat them on the same bet; a portfolio that is volatile but different wins in the scenarios where their theme falls behind. The search found that the second route gives more expected prize money.

Why not trade actively during the competition? Tested. Fixed rules gave no gain, and the weekly rollout acts only when an alternative beats doing nothing by more than twice its

uncertainty. In backtests that happens about once in five competitions. Every trade costs €5 plus 0.35%.

What would you improve with more time? A conditional model of crash correlations, real competitor trading behaviour instead of buy-and-hold, and options data for European stocks.

Did you write all the code yourself? Answer honestly: the code was written with an AI assistant, while you set the goal, the constraints and the validation rules, ran and checked every step, and made the design decisions. What matters is that you can explain and defend each part, which is what this paper is for.